LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

Progress meeting 2: what we built, what it found, what we measured

CS-5187 term project A3 · 29 October 2026

Where we were last time

The question and the three claims from meeting 1.

State on 16 October

- Harness, typed grammar and random baseline built
- 100% valid statements on 300 test cases
- No bug candidates, no campaign run
- LLM loop only planned

Three claims

- One tree, four jobs: the derivation tree generates, mutates, is rewritten by oracles, and is reduced
- Soundness we measure: validity and oracle precision are reported numbers
- The LLM proposes, the server decides: a proposal counts only if it runs and reaches changed code

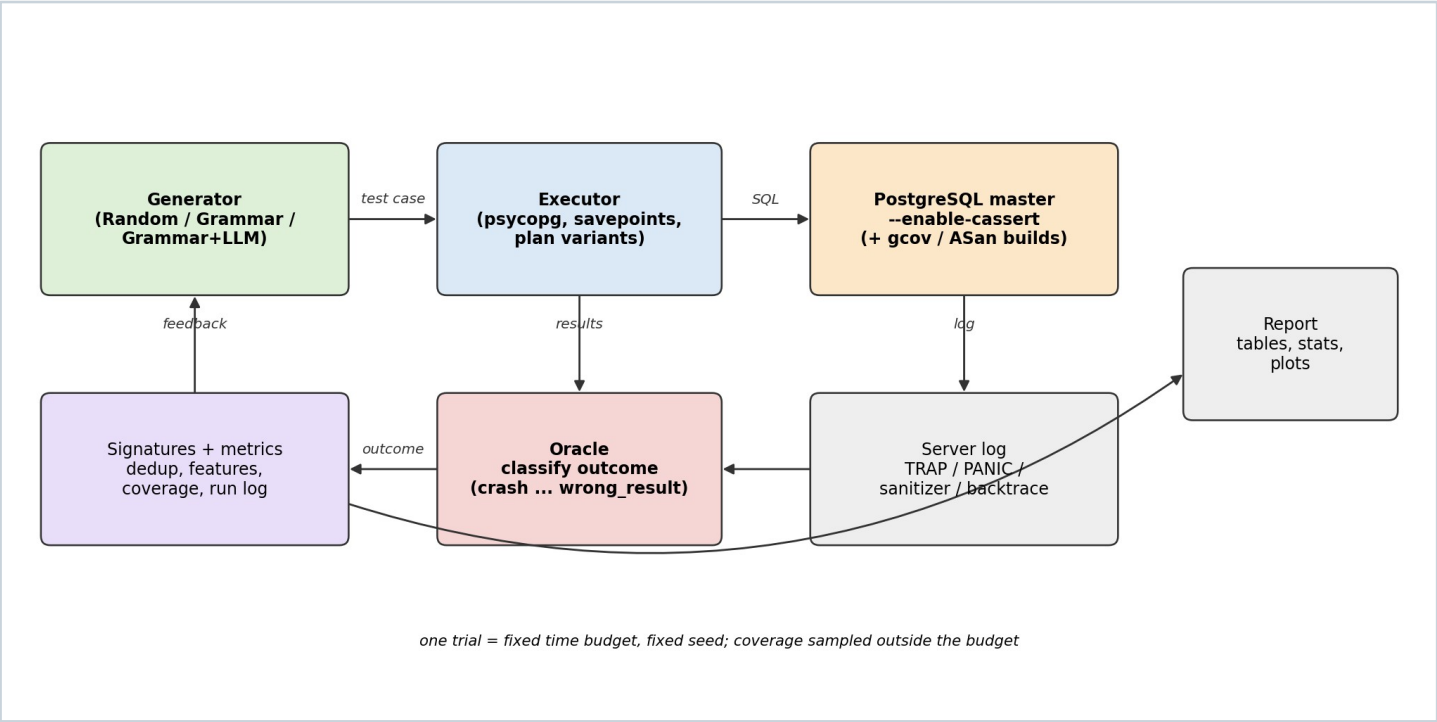
Plan versus what we did

Every promise from meeting 1 has an outcome.

Meeting 1	Now	
Typed grammar, 3 oracles	78 rules, 12 oracle kinds, 18 plan variants, cross-version	done
LLM loop: mine, propose, gate	Built; run on 10 commits, 4 accepted	done
8-12 re-injected bugs	23 reverted fixes, 20 observable	exceeded
10 trials per arm	5 trials x 1 h x 7 arms	reduced
At least one bug acknowledged	9 bugs found, 4 answered, 2 fixed upstream	met
0 bug candidates so far	93 candidates triaged, each with a verdict	done

Build 1: a typed grammar that keeps the tree

Queries are trees of typed nodes, not strings.



78 rules · 421 alternatives · 13 types

In short

- Each node has a type and a scope
- Changing a node keeps the SQL valid
- Weights favour alternatives that reach new plans
- Literals come from the data, including NaN and -0

Build 2: oracles that know the right answer

To see wrong results, compare a query with something that must give the same rows.

Idea: ternary logic partitioning

- Q = SELECT … WHERE p
- Every row: p is true, false or NULL
- Q must equal Q(p) + Q(NOT p) + Q(p IS NULL)
- A difference means a bug

These checks are why we can find bugs that never crash.

Oracle	Compares
TLP, NoREC	query vs partitioned or counted form
Expression rewrites	e vs (SELECT e), COALESCE(e,e)
Row-level security	restricted role vs policy inlined
18 plan variants	same query, other planner settings
Cross-version	master vs release 18.6
Crash / assertion / error	server log and error class

First wrong-result bug: range containment with NULL

The planner folds the call to a constant, so a NULL element gives true.

```
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$ PSQL=~/pgfuzz/install/REL_18_STABLE/bin/psql
for p in 5435 5440; do
  $PSQL -h 127.0.0.1 -p $p -U postgres \
    -c "SELECT version()" \
    -c "SELECT c, '(,)'::int4range @> c AS a, 'empty'::int4range @> c AS b, NOT ('empty'::int4range @> c) AS d FROM (VALUES (NULL::int), (1)) v(c);" \
    -c "EXPLAIN VERBOSE SELECT '(,)'::int4range @> c FROM (VALUES (NULL::int), (1)) v(c);"
done
                                                version

--------------------------------------------------------------------------------
--------------------------
 PostgreSQL 18.6 on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubunt
u2~24.04.1) 13.3.0, 64-bit
(1 row)

 c | a | b | d
---+---+---+---
   | t | f | t
 1 | t | f | t
(2 rows)

                        QUERY PLAN
-----------------------------------------------------------
 Values Scan on "*VALUES*"  (cost=0.00..0.03 rows=2 width=1)
   Output: true
(2 rows)

                                                version

--------------------------------------------------------------------------------
-----------------------------
 PostgreSQL 20devel on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ub
untu2~24.04.1) 13.3.0, 64-bit
(1 row)

 c | a | b | d
---+---+---+---
   | t | f | t
 1 | t | f | t
(2 rows)

                        QUERY PLAN
-----------------------------------------------------------
 Values Scan on "*VALUES*"  (cost=0.00..0.03 rows=2 width=1)
   Output: true
(2 rows)
```

What it shows

- A NULL row should give NULL in every column
- 18.6 and master print true
- EXPLAIN shows Output: true: the call became a constant
- Present in released 17 and 18
- Found by the e versus (SELECT e) rewrite oracle
- Patch with regression tests sent

Everything we found upstream

Nine bugs, all found with our fuzzers and oracles, all reproduced on an assertion-enabled build.

Bug	Upstream	Status
JSON_TABLE collations (wrong result, assertion)	#19768	filed, awaiting reply
Range containment with NULL (wrong result)	#19793	filed with patch
MERGE into partitioned table: SubPlan error	#19802	confirmed, fix posted
Parallel Append assertion	#19803	confirmed, fixed on master
to_char NaN crash in NORMALIZE	#19771	fix committed
NOT IN anti-join ignores a cast (wrong result)	pgsql-bugs	filed
IGNORE NULLS window frame hang	pgsql-bugs	developer replied
Assertion on duplicate extended-stats expression	pgsql-bugs	filed
Non-finite MCV frequencies in extended stats	pgsql-hackers	patch submitted

9 filed · 4 answered · 2 fixed

The other bugs in a few words

Short notes for the bugs that are not covered in the deep dives.

Bug	What it is
#19768 JSON_TABLE	Never assigns collations to its context item and PASSING arguments. Assertion on master, wrong result on 18.6. One-line patch.
#19793 range containment	Planner folds a strict operator to a constant, so NULL gives true. Wrong result in 17, 18 and master.
NOT IN anti-join	A regression: the conversion ignores an implicit cast. In 19rc1 and master, not 18.6. Patch and test.
IGNORE NULLS hang	A frame scan of a window function can hang.
dependencies.c assertion	A duplicate expression in extended statistics fails an assertion. Reproducer and patch.
Non-finite MCV	pg_restore_extended_stats accepts non-finite frequencies. Patch submitted.

Deep dives

Three bugs from the campaigns, explained step by step on the next slides.

Bug	Found by	Status
#19802 MERGE: SubPlan with no parent plan	grammar campaign	**confirmed, fix posted**
#19803 parallel Append assertion	LLM-fragment arm + TLP check	**confirmed, fixed on master**
#19771 to_char NaN in NORMALIZE	grammar campaign, reduced to 27 tokens	**fix committed**

Triage: reproduce, shrink, localize

A raw finding is a long query; maintainers need a small reproducer.

```
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$ cat ~/cs5187-term-project/runs/memberA-rq3/logs/A3_triage.txt
[cde9712bc8db] assertion via assertion: reproducing x3
  statements: 3 -> 2
  statements: 2 -> 1
  clauses: 572 -> 570 tokens
  clauses: 570 -> 543 tokens
  clauses: 543 -> 529 tokens
  clauses: 529 -> 527 tokens
  clauses: 527 -> 515 tokens
  items: 515 -> 420 tokens
  items: 420 -> 410 tokens
  items: 410 -> 318 tokens
  items: 318 -> 314 tokens
  items: 314 -> 290 tokens
  items: 290 -> 277 tokens
  items: 277 -> 273 tokens
  items: 273 -> 269 tokens
  items: 269 -> 265 tokens
  items: 265 -> 263 tokens
  items: 263 -> 261 tokens
  groups: 261 -> 201 tokens
  groups: 201 -> 162 tokens
  groups: 162 -> 141 tokens
  groups: 141 -> 139 tokens
  groups: 139 -> 125 tokens
  groups: 125 -> 116 tokens
  reduced to 1 statement(s), 116 tokens in the culprit (406 runs, 73.3 s)
1 candidate(s) triaged into ../results/triage_demo; verdict template: ../results/triage_demo/verdicts.template.yaml
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$
```

```
bhuvi@BlazingBhuvi:/mnt/c/Users/Bhuva/OneDrive/Desktop/cs5187-term-project-main$ cat ~/cs5187-term-project/runs/memberA-rq3/triage_demo/cde9712bc8db/upstream_report
.txt 2>&1
Subject: fails an assertion -- <one line: the feature involved>

PostgreSQL version: 20devel (built with --enable-cassert)
Also checked on: <latest release: run the script there too>
Platform: <OS, compiler>

Description
-----------

The following script fails an assertion.

TRAP: failed Assert("list_length(rcexpr->inputcollids) == nopers"), File: "execExpr.c", Line: 2078

Script
------

-- pgfuzz triage of cde9712bc8db: assertion via assertion
SELECT CASE CASE WHEN NULL THEN NULL ELSE (SELECT MAX ('{NULL}'::text[]) FROM JSON_TABLE (CASE WHEN (ROW (10.0::numeric, 10) >= ROW (0.0::numeric, 1) IS FALSE) THEN
 '{"k": -1, "a": [1, 2, 3]}'::jsonb ELSE '{"k": 1, "a": [1, 2]}'::jsonb END, '$.a[*]' COLUMNS ( c1 int PATH '$')) AS a6) END WHEN NULL THEN NULL WHEN MAX ('{a,b,c,d
}'::text[]) OVER (PARTITION BY '{1,2,3}'::int[]) THEN NULL ELSE '{NULL,1}'::int[] END AS c0 FROM (SELECT a1.c0 AS c0 FROM t0 AS a1) AS a2;

Expected
--------

<what should happen, and why: SQL standard / documentation section>

<If bisected: first bad commit (scripts/bisect_pg.sh).>
bhuvi@BlazingBhuvi:/mnt/c/Users/Bhuva/OneDrive/Desktop/cs5187-term-project-main$
```

590 → 79

tokens after tree + token reduction

x3

reproduced on a fresh connection

report

draft upstream report is generated

Precision: strong oracles also raise false alarms

Every one of 93 candidates got a verdict.

Verdict	Count
real, new	7
real, also reported upstream	15
false positive	37
flaky	28
to verify	6

0.16

precision

0.37

if duplicates count

Ten false-alarm classes, all fixed

- Six found during development
- Equal intervals printed differently: 16
- ORDER BY NULL in ordered aggregates: 2
- Cross-version state divergence: 5
- LLM rewrite that changed the collation: 1

The LLM extension and its gate

The LLM proposes grammar pieces from recent commits; the server decides.

```
gow@Gowtham:~/cs5187-term-project/runs/memberB-rq1-llm$ tail -40 logs/B2_extend.txt
| finished | 9 |
| accepted | 4 |
| acceptance_rate | 0.444 |
| accepted_fragments | 4 |
| accepted_by_round | {"1": 2, "2": 1, "3": 1} |
| rounds_total | 24 |
| rejections_by_stage | {"validity": 15, "parse": 2, "coverage/seeds": 1, "reachability": 2} |
| tokens | {"input_tokens": 312197, "output_tokens": 156417, "cache_read_input_tokens": 46568, "cache_creation_input_tokens": 0} |
| cost_usd | 0.0 |
| mean_validity | 0.376 |
| coverage_gain_lines | 34 |
| gate_bug_candidates | 0 |
| rewrites_proposed | 7 |
| rewrites_accepted | 7 |
| rewrites_with_mismatch | 0 |
18:30:10 INFO d4c15309e7 round 1: rejected
  d4c15309e7-r2.json: gemini-3-flash-preview, 15659 in / 14514 out, 73s

=== extend invocation 15 (18:31:22) ===
1 commit(s) finished, 0 accepted; ledger: ../results/llm/ledger.jsonl
| metric | value |
|---|---|
| commits | 10 |
| finished | 10 |
| accepted | 4 |
| acceptance_rate | 0.4 |
| accepted_fragments | 4 |
| accepted_by_round | {"1": 2, "2": 1, "3": 1} |
| rounds_total | 25 |
| rejections_by_stage | {"validity": 16, "parse": 2, "coverage/seeds": 1, "reachability": 2} |
| tokens | {"input_tokens": 327856, "output_tokens": 170931, "cache_read_input_tokens": 46568, "cache_creation_input_tokens": 0} |
| cost_usd | 0.0 |
| mean_validity | 0.365 |
| coverage_gain_lines | 34 |
| gate_bug_candidates | 0 |
| rewrites_proposed | 7 |
| rewrites_accepted | 7 |
| rewrites_with_mismatch | 0 |
18:32:01 INFO d4c15309e7 round 2: rejected
=== extend finished: no prompt left ===
```

Gate checks

- Fragment parses
- Used at least 5 times
- 60% of statements run
- Seeds run
- More changed lines covered

Outcome

- 10 commits, 4 accepted
- 16 of 25 rounds failed on validity
- One rewrite changed the collation; now rejected

Ground truth: 23 real bugs put back

To compare arms we need bugs we know exist.

```
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$ python3 -m pgfuzz bench verify configs/rq3.yaml
| bug | commit | symptom | all injected | evidence |
|---|---|---|---|---|
| rb-11b56ee2 | 11b56ee2 | assertion | assertion | pathkeys.c: var->varno == rel->relid |
| rb-94e1385e | 94e1385e | wrong_result | wrong_result | stmt 0: 1 rows on fixed, 1 on buggy |
| rb-41180127 | 41180127 | missing_error | missing_error | stmt 0: fixed raises 22030 duplicate JSON object key value |
| rb-b28a2b2b | b28a2b2b | wrong_result | wrong_result | stmt 5: 1 rows on fixed, 1 on buggy |
| rb-f5635c27 | f5635c27 | none | none |  |
| rb-d8444163 | d8444163 | wrong_result | wrong_result | stmt 3: 0 rows on fixed, 1 on buggy |
| rb-62e46c00 | 62e46c00 | error | error | stmt 2: 42P10 there is no unique or exclusion constraint matching the ON CONFLICT specific |
| rb-738d19d4 | 738d19d4 | internal_error | internal_error | stmt 6: XX000 unsupported join alias expression |
| rb-6b858a36 | 6b858a36 | assertion | assertion | subselect.c: !IsA(node, SubPlan) |
| rb-8b29b8da | 8b29b8da | internal_error | internal_error | stmt 10: XX000 could not find pathkey item to sort |
| rb-d764d61a | d764d61a | wrong_result | wrong_result | stmt 7: 1 rows on fixed, 1 on buggy |
| rb-dfcf6fa7 | dfcf6fa7 | internal_error | internal_error | stmt 6: XX000 could not find pathkey item to sort |
| rb-a464d31f | a464d31f | assertion | assertion | relnode.c: join_clause_is_movable_into(rinfo, baserel->relids, joinrelids) |
| rb-852fd5b8 | 852fd5b8 | internal_error | internal_error | stmt 5: XX000 no relation entry for relid 3 |
| rb-a8bb1a77 | a8bb1a77 | assertion | assertion | rewriteManip.c: context->new_index != INVALID_VAR |
| rb-9d0adee8 | 9d0adee8 | none | none |  |
| rb-895c1b5f | 895c1b5f | none | none |  |
| rb-35d3f206 | 35d3f206 | internal_error | internal_error | stmt 21: XX000 variable not found in subplan target list |
| rb-1bf74c80 | 1bf74c80 | wrong_result | wrong_result | stmt 4: 4 rows on fixed, 3 on buggy |
| rb-7774249d | 7774249d | wrong_result | wrong_result | stmt 1: 1 rows on fixed, 1 on buggy |
| rb-0d995793 | 0d995793 | wrong_result | wrong_result | stmt 4: 0 rows on fixed, 1 on buggy |
| rb-d6eac691 | d6eac691 | wrong_result | wrong_result | stmt 0: 0 rows on fixed, 2 on buggy |
| rb-3b120b1e | 3b120b1e | missing_error | missing_error | stmt 0: fixed raises 22003 integer out of range |
note: rb-94e1385e raises errors on the fixed server: stmt 1: 2201E cannot take logarithm using base one; stmt 2: 2201E cannot take logarithm using base one
note: rb-41180127 raises errors on the fixed server: stmt 0: 22030 duplicate JSON object key value
note: rb-6b858a36 raises errors on the fixed server: stmt 3: 21000 more than one row returned by a subquery used as an expression
note: rb-3b120b1e raises errors on the fixed server: stmt 0: 22003 integer out of range; stmt 1: 22003 integer out of range; stmt 2: 22003 integer out of range

20/23 bugs observable; copy the symptoms into /home/bhuvi/cs5187-term-project/src/bench/reinjected/bugs.yaml
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$
```

In short

- Real bugs, not synthetic ones
- Fixed, solo and injected servers
- 8 wrong results, 5 internal errors, 4 assertions, 3 error cases
- Only 9 of 20 visible without an oracle

23 → 20

reverted fixes → observable

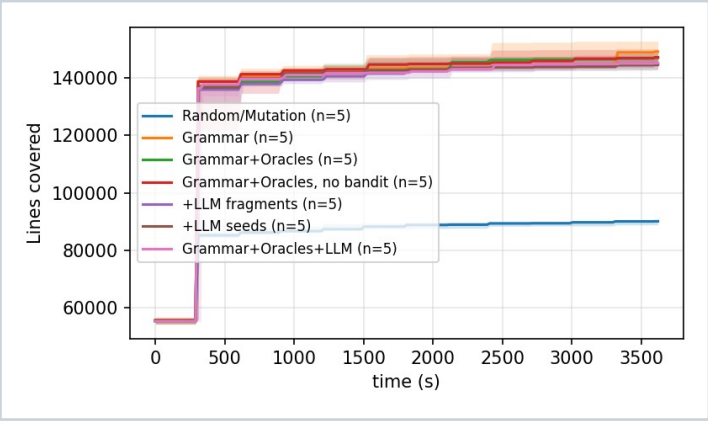
9 of 20

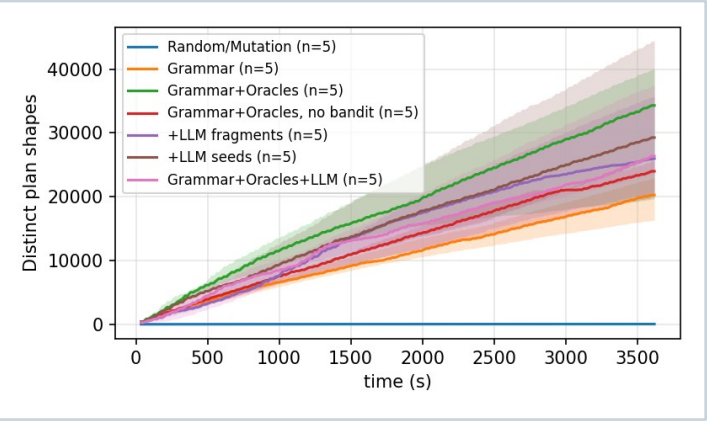
visible to a crash-only fuzzer

Results: reach of the typed grammar

7 arms, 5 trials of 1 hour each, same server, schema, oracles and seeds.

Arm	Valid	Lines	Features	Plan shapes	Tests
Random / mutation	**26.0%**	22.2%	23	43	1,074k
Grammar	**99.3%**	**36.7%**	101	20,279	71k
Grammar + oracles	96.3%	36.2%	101	**34,313**	59k
+ LLM	95.3%	35.7%	106	26,375	47k





Reading

- Grammar beats random on every metric
- Oracles: more plan shapes, no more lines
- LLM: +5 features, no more lines

Results: re-injected bugs

Few of the 23 are found in one hour; we say so.

Bug	Random	Grammar	+Oracles	+LLM
dfcf6fa7 (internal error)	0/5	5/5	4/5	5/5
11b56ee2 (assertion)	0/5	0/5	1/5	1/5
8 wrong-result bugs	0	0	0	0
Bugs found of 20	0	1	2	2



Why so few

- One hour is short
- 5 trials cannot separate rare finds
- Strict attribution to solo servers

What the numbers tell us

Three honest findings.

Typing pays

- 99% vs 26% valid statements
- 36.7% vs 22.2% of lines
- random runs 15x more tests

Oracles find hidden bugs

- real wrong results in released versions
- cost: precision 0.16
- ten false-alarm classes fixed

The LLM needs a gate

- +5 features, no extra coverage
- 4 of 10 commits usable
- a tested rewrite is not a proven rewrite

What went wrong and how we handled it

Deviations from the plan, reported as they happened.

Infrastructure

- A worker's server failed crash recovery: 14 trials re-run, same seeds
- Laptop sleep: budget unchanged

Substitutions

- LLM arm used Gemini Flash (free tier)
- 10 of 25 commits (daily quota)
- 5 trials instead of 10

Measured after the fact

- Four false-alarm classes fixed after the campaigns and tested on stored reproducers; not re-measured by a new campaign

Next steps to 13 November

Verification, follow-up and the paper.

When	What
30 Oct to 5 Nov	Verify 6 open candidates; follow upstream threads; re-run reproducers
2 to 6 Nov	Meeting 3: full result tables, paper draft
12 Nov	Presentation
13 Nov	Paper and repository due

Questions for you

- Do committer-confirmed bugs (one fixed) meet the top band even though the benchmark shows no oracle advantage?
- Is it acceptable to argue precision after the fixes from stored reproducers?

Thank you. Questions?

github.com/PASS-IITG/cs5187-term-project-blazing-bhuvi